COS214_Project

TEAM DETAILS AND GITHUB REPOSITORY LINK:

GitHub repository link: https://github.com/reneilwemolopyane/COS214-Prac6.git

Team Details

STUDENT NAME AND SURNAME	STUDENT NUMBER
Lizalise Mbonisweni	u23587874
Boikemelo Masoka	u25128648
Thembelisha Skosana	u25224663
Lethabo Molobi	u25090209
Boitumelo Monareng	u25208943
Reneilwe Molopyane	u25161874
Dingalethu Ngumbela	U25170547

TASK 1: Workflow-management research findings and references

1. Workflow Basics: Definition vs Instance

A Workflow Management System defines, manages and executes workflows based on their logic. A workflow definition is the blueprint of steps, rules and participants for a process. Each new published version is used by new instances, while active instances finish on the version they started with. A workflow instance is one running execution of that definition.

2. Work Structure

Work items can be grouped into stages or sub-processes. They can depend on each other, run in parallel and join again. Items move through states such as available, assigned, running, completed or suspended, so TIVIDY must track each item's current state and what can happen next.

3. Assignment and Routing

Resources (people or equipment) are modelled separately from the process. Tasks name a role, and a specific person is chosen only at runtime, so one definition serves many instances (Russell et al., 2004). Allocation can be role-, capability- or history-based, enforce separation of duties, or pick by round robin or shortest queue. Items can be pushed to a resource or pulled by one. Delegation and deadline-based escalation can also change who does the work. For TIVIDY, roles, allocation rules and deadlines belong to the definition. Assigned participants, workload and assignment history belong to each instance.

4. Approvals, Rejection and Escalation

Approval is a normal paused state that ends in approval, rejection or escalation (Temporal Technologies, 2026). Steps can be sequential or parallel and may need all approvers, a quorum or any one approver (django-workflow-kit, n.d.). A rejection stops later steps and records a reason (ServiceNow, n.d.). Timeouts trigger re-notification, then the manager, then a default decision (EmpowerID, n.d.). For TIVIDY, approval modes and escalation chains belong to the definition. Status, reasons and escalation history belong to each instance.

5. Events and Failures

Russell, van der Aalst and ter Hofstede (2006) identify five exception types: item failure, deadline expiry, resource unavailability, external triggers and constraint violations. Each is handled at item level, case level and through recovery such as rollback or compensation. Engines retry temporary failures, reroute business errors and raise incidents when retries run out (Camunda, 2025). For TIVIDY, handling rules belong to the definition. Retries, errors and incidents belong to each instance.

6. Integration and Reusability

Engines connect to external systems through defined interfaces (Workflow Management Coalition, 1995) and connectors (Camunda, 2025). Connections are configuration, not code, and can be reused across flows (Microsoft, 2024). For TIVIDY,the connector reports whether a failure is temporary, and the engine decides what to do, which makes our scenario one use of a reusable engine.

7. History, Monitoring and Versioning

History is an append-only log of runtime events that supports recovery, replay and auditing. It is kept separate from the runtime database so performance is not affected. Versioning lets running instances finish on their original version while new ones use the update. Monitoring shows live instances so administrators can spot bottlenecks and see who is handling each item.

REFERENCES

1. Camunda (2025) *Glossary*. Camunda 8 Docs. Available at: https://docs.camunda.io/docs/reference/glossary
2. Camunda (2025) *Dealing with problems and exceptions*. Camunda 8 Docs. Available at: https://docs.camunda.io/docs/components/best-practices/development/dealing-with-problems-and-exceptions/
3. Camunda (2025) *Versioning process definitions*. Camunda 8 Docs. Available at: https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/
4. Koksal, P., Arpinar, S.N. and Dogac, A. (1998) 'Workflow history management', *ACM SIGMOD Record*, 27(1), pp. 67–75. Available at: https://www.srdc.com.tr/share/publications/1998/sonsig.pdf
5. Microsoft (2024) *Use connector actions in desktop flows*. Microsoft Learn. Available at: https://learn.microsoft.com/en-us/power-automate/desktop-flows/how-to/use-connector-actions
6. Rolfe, P.A. (2001) *Code versioning in a workflow management system*. Doctoral dissertation. Massachusetts Institute of Technology.
7. Russell, N., ter Hofstede, A.H.M., Edmond, D. and van der Aalst, W.M.P. (2004) *Workflow Resource Patterns*. Technische Universiteit Eindhoven. Available at: https://pure.tue.nl/ws/files/1984893/591788.pdf
8. Russell, N., van der Aalst, W.M.P. and ter Hofstede, A.H.M. (2006) 'Workflow exception patterns', in *Advanced Information Systems Engineering (CAiSE 2006)*. Lecture Notes in Computer Science, vol. 4001. Berlin: Springer, pp. 288–302. Available at: https://doi.org/10.1007/11767138_20
9. Schiefer, J., Jeng, J.J. and Bruckner, R.M. (2003) 'Real-time workflow audit data integration into data warehouse systems', in *ECIS 2003*, pp. 1697–1706. Available at: https://www.ifs.tuwien.ac.at/~js/download/ecis2003.pdf
10. Workflow Management Coalition (1995) *The Workflow Reference Model*. Document TC00-1003. Available at: http://www.workflowpatterns.com/documentation/documents/tc003v11.pdf
11. Zur Muehlen, M. (2004) *Workflow-based process controlling: foundation, design, and application of workflow-driven process information systems*. Advances in Information Systems and Management Science, vol. 6. Berlin: Logos Verlag. Available at: https://www.logos-verlag.de/cgi-bin/buch/isbn/0388

TASK 2: DEFINING THE SCENARIO AND SYSTEM

TIVIDY is a generic workflow management system. It stores descriptions of processes, creates independent running copies of them, coordinates the work inside each copy, and keeps a record of everything that happens. It is not tied to any particular organisation or industry.

Main responsibilities:

- **Define workflows:** store workflow definitions made of stages, work items, sub-processes, dependencies and rules, and keep older versions of definitions.
- **Create and run instances:** start independent workflow instances from a definition, each with its own data, participants, progress and history.
- **Manage work items:** track each work item through its lifecycle (created, available, assigned, started, completed, rejected, cancelled, escalated) and only allow operations that are valid in the current state.
- **Route and assign work:** decide what becomes available next, and who receives each work item, using rules that can be changed.
- **Handle approvals and escalation:** support single and multi-level approvals, rejections and deadlines, and escalate when no one responds.
- **React to events:** respond to internal events (a work item completes) and external ones (a document arrives, a deadline passes), and notify any interested parts of the system.
- **Communicate with external systems:** exchange information with outside systems through a controlled boundary.
- **Record history:** retain actions and state changes so the system can explain what happened, and support undo or rollback where it makes sense.
- **Monitor and report:** provide views of progress, delays and outcomes without placing every report inside the core model.
-

Boundaries (What TIVIDY does not do)

- It coordinates work but does not perform the work itself. People and external systems carry out tasks, and TIVIDY tracks and routes them.
- It does not contain the business rules of any one organisation. Rules are supplied through configurable definitions.
- It does not own external systems. It communicates with them through connectors that translate between TIVIDY and their interfaces.
- It does not decide what a process should be. It runs what the definition describes.

Definition vs running instance

Belongs to the workflow definition	Belongs to a running instance
Stages, work items and sub-processes	Current stage and state of each work item
Dependencies and ordering rules	Which dependencies have been satisfied
Roles allowed to perform each item	The specific participants assigned
Routing, assignment and approval rules	Decisions actually made along the way
Deadlines and escalation rules	Actual start times, due times, escalations triggered
Version number	The definition version it was started from
Data requirements (what must be collected)	The actual data values and attachments
	History of actions and state changes

Changing one instance must never change the definition or any other instance

Parts expected to vary

- How work is assigned (by role, workload, or a rule calculated from data)
- How approvals work (one person, several levels, escalation)
- The lifecycle behaviour of work items, and extra behaviour layered on (validation, priority, auditing, notifications, security)
- The structure of work (sequential, parallel, nested)
- Notifications and event reactions, since new interested parties should be addable
- External integrations, since new systems should be connectable without redesigning the engine
- Reports and views of running workflows
- How workflows are built (different ways of constructing definitions)

Design rationale

Process descriptions are kept separate from running processes so that many instances can run from one definition without interfering. The parts listed as varying are isolated behind stable abstractions, so a new rule, integration or report can be added without rewriting the engine or the work-item classes.

For the final project TIVIDY runs two workflows from two different industries (insurance and education) on the same engine. Everything specific to an industry, such as role

names, approval levels and limits, work items, decorator stacks and adapters, is supplied as configuration in `main.cpp` or data files. The pattern classes in the engine contain no industry names. Neither scenario defines TIVIDY.

Industry 1: Insurance (Motor Vehicle Claim)

1. Organisation

- A short-term insurance company that handles motor vehicle claims, with a call centre, claims department, assessors, a fraud unit and approved panel beaters.

2. Process being managed

- A motor vehicle accident claim, from the moment a policyholder reports an accident until the car is repaired and paid for, or the claim is rejected.

3. Participants and roles

- **Policyholder:** reports the claim and uploads documents
- **Claims handler:** checks the claim and manages it end to end
- **Assessor:** inspects the damaged car and estimates the cost
- **Panel beater:** quotes for and repairs the car
- **Claims manager / senior manager:** approve claims above certain amounts
- **Fraud investigator:** reviews suspicious claims
- **Finance clerk:** releases payment

4. Major pieces of work

- Lodge claim → verify policy → collect documents (police case number, photos, driver's licence, quotes) → assess damage → decide → authorise repair → repair → final inspection → pay → close

5. Decisions and routing points

- Is the policy active and does it cover this type of accident?
- Is the car repairable or a **write-off**? A write-off leads to a cash payout instead of a repair.
- Does the claim score as **suspicious**? If so, it is routed to the fraud investigator.
- How much is it? The amount decides which approval level is needed.

6. Approvals and dependencies

- Approval levels by amount, configured as a chain: Claims Handler up to R20 000, Manager up to R100 000, Senior Manager with no upper limit. The chain is built from configuration, so the limits can change without changing the engine.
  Parallel work: the police report check, assessor inspection and repair quotes happen at the same time, and the decision waits for all three.
- Repair can't start until approval is given; payment can't happen until the final inspection passes.

7. Failures and exceptions

- Policy lapsed or excess not paid: claim rejected.
- Documents missing: request sent to the policyholder; escalate if there's no response in 7 days.
- Approver doesn't respond in time: **escalate** to the next level.
- Fraud confirmed: claim rejected, and any payments already made are reversed (**compensation**).
- Payment gateway or police system down: **retry**, then raise an **incident** for a person to fix.
- Repair fails the final inspection: send back to the panel beater (**loop**).
- A level that does not respond before its deadline is skipped and the request moves to the next level, which receives a fresh deadline.

8. External systems

- Policy administration system (check cover)
- Police case verification, using the case number
- Vehicle valuation service (write-off decisions)
- Panel beater portal (quotes and repair status)
- Payment gateway / bank
- SMS and email notifications

9. Expected outcome

- **Approved:** car repaired, or write-off paid out; policyholder notified; claim closed.
- **Rejected:** policyholder notified with the reason; claim closed.
- In both cases, a full **audit history** of every decision and action is kept.

10. Input data

Policy number, claim amount, accident description, police case number, photos, driver’s licence, repair quotes, fraud score.

Industry 2: Education (Bursary Application)

1. Organisation

A university's financial aid office, which awards funded bursaries to students.

2. Process being managed

A bursary application, from the moment a student applies until the bursary is paid and the application is closed, or the application is rejected or waitlisted.

3. Participants and roles

Student: applies and uploads documents

Bursary officer: checks the application and the references

Registrar's office: confirms registration and academic record

Financial aid assessor: performs the means test

Faculty officer, bursary committee, dean and finance director: approve at increasing amounts

Finance clerk: releases payment

4. Input data

Student number, requested amount, academic average, household income, documents (ID, proof of income, motivation letter, reference letter) and the bursary type (merit or need).

5. Major pieces of work

Submit application → verify registration → collect documents → academic record check, means test and reference check (in parallel) → shortlist or reject → approve by amount → award letter and acceptance → release payment → notify student → close application

6. Decisions and routing points

Is the student registered and in good standing? If not, the application is rejected.

Is the average above the threshold, or does the means test show need? If not, the application is rejected.

Merit and need bursaries take different routes.

How much was requested? The amount decides which approval level is needed.

Is the bursary fund exhausted? If so, the student is waitlisted.

7. Approvals and dependencies

Approval levels by amount: Faculty Officer up to R10 000, Bursary Committee up to R50 000, Dean up to R150 000, Finance Director with no upper limit.

Parallel work: the academic check, means test and reference check happen at the same time, and the shortlisting decision waits for all three.

Payment cannot be released until the bursary is approved and the student has accepted the award.

8. Failures and exceptions

Documents missing: a request is sent to the student, and it is escalated if there is no response in 7 days.

An approver does not respond in time: the request escalates to the next level.

Rejected: the reason is recorded and the student is notified.

Document fraud found after payment: the award is cancelled and the payment is reversed (compensation).

Payment gateway down: retry, then raise an incident for a person to fix.

Student deregisters before payment: the award is cancelled.

9. External systems (stub adapters)

Student records system (registration and academic record)

Income verification service

Payment gateway / bank

SMS and email notifications

10. Expected outcome

Approved: the bursary is paid, the student is notified and the application is closed.

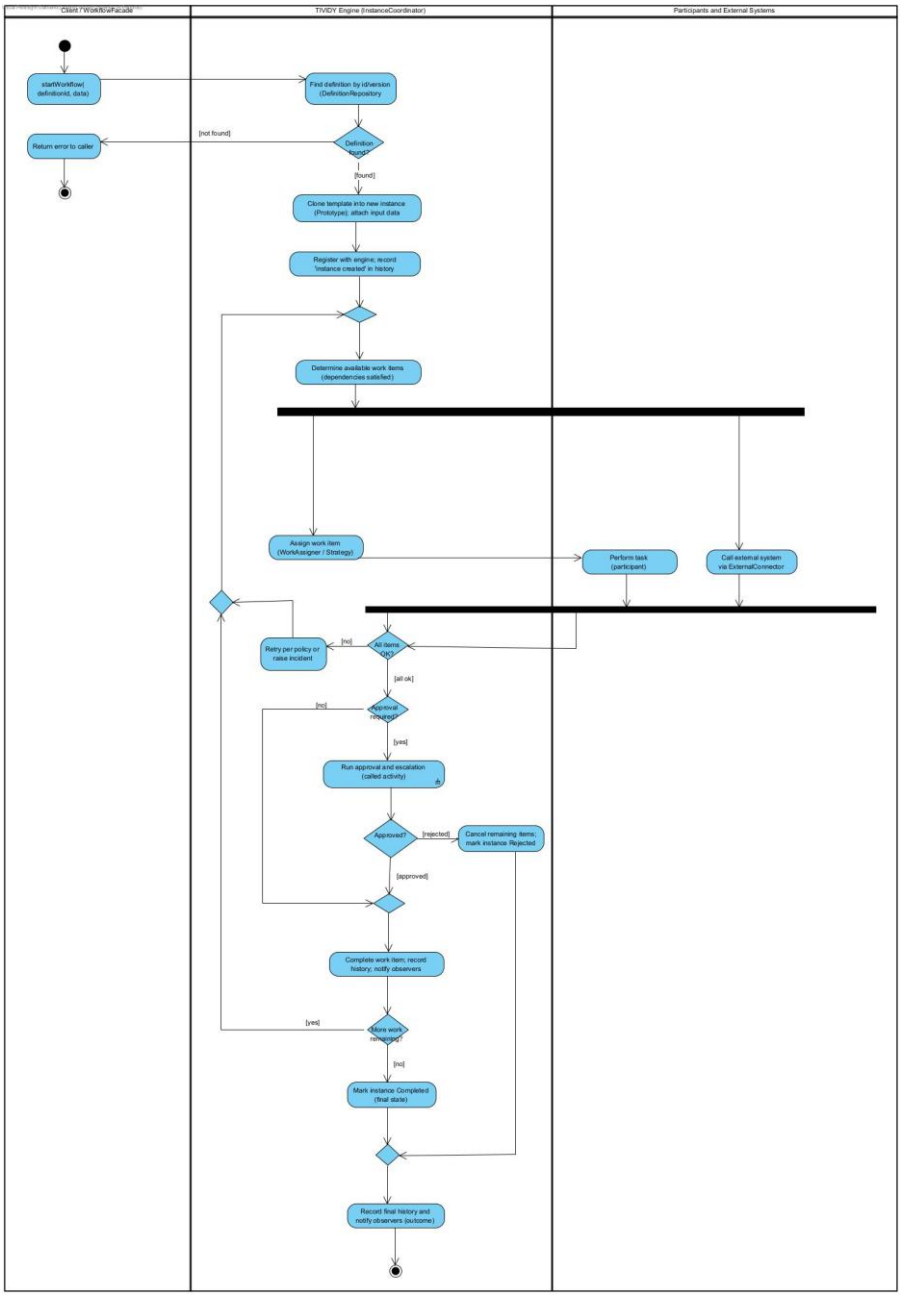
Rejected or waitlisted: the student is notified with the reason, and the application is closed.

In all cases, a full audit history of every decision and action is kept.

Task 3 - The three UML Activity Diagrams

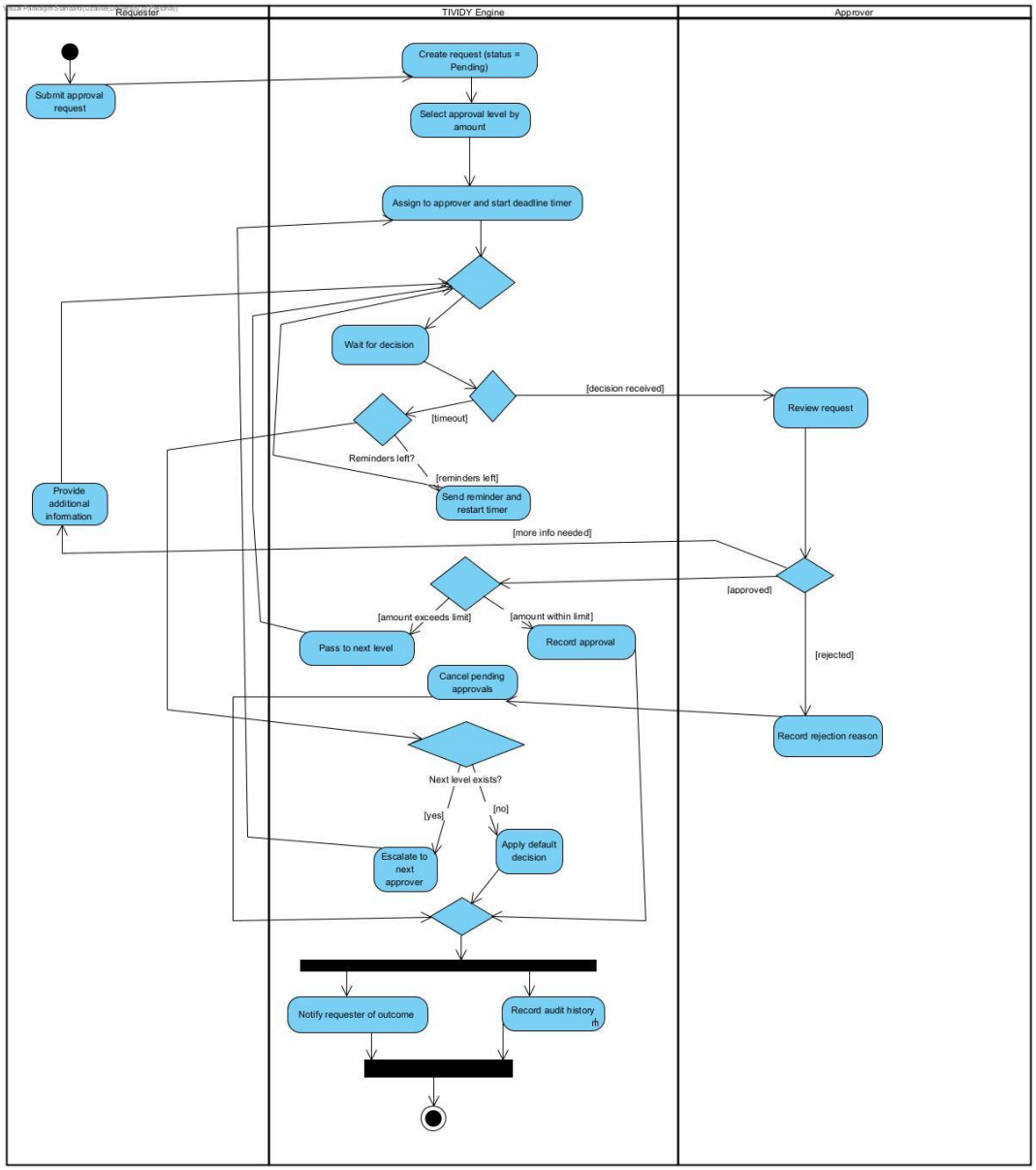
1. A **generic workflow execution** showing the progression of a workflow instance from creation or start until completion or another appropriate final outcome.

Progress of a workflow instance from startWorkflow() to Completed or Rejected. Generic because there is no insurance logic.



2. Approval and Escalation Process

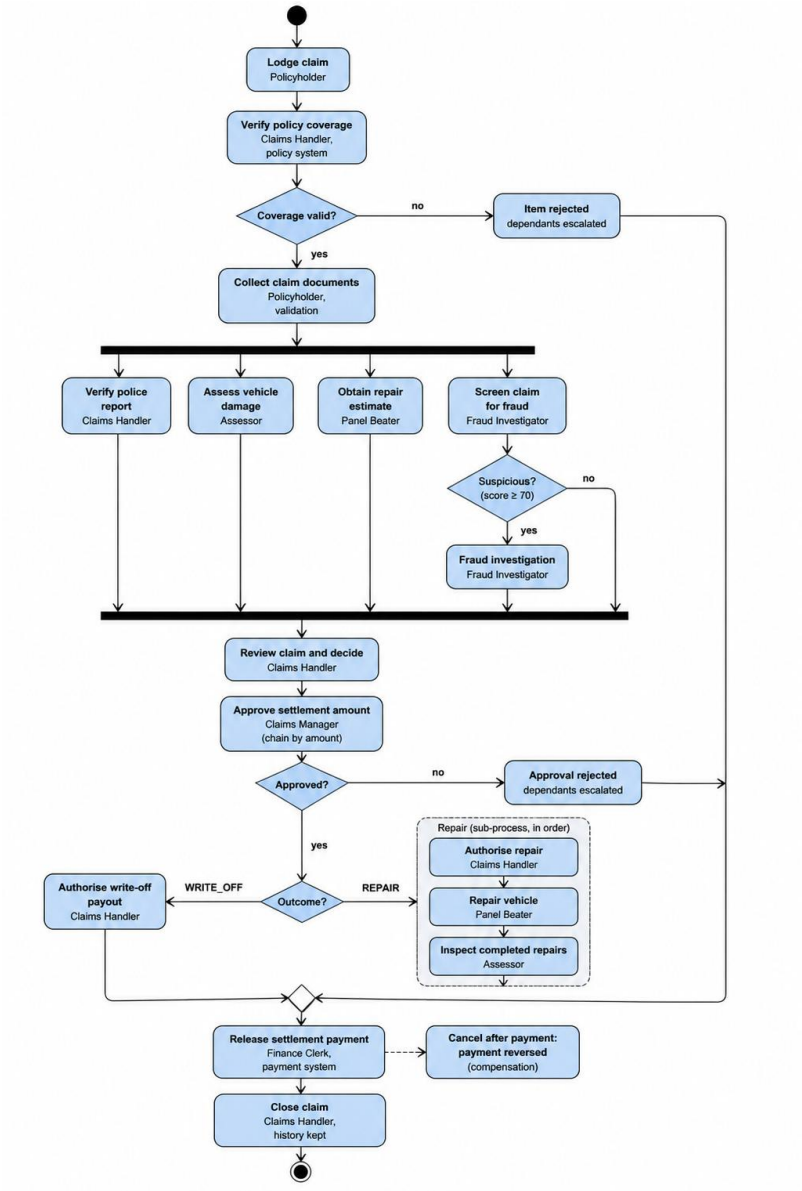
How a request moves from submission through review to approval, rejection or escalation. It covers multi-level approval by amount, deadline timers, reminders, requests for more information and default decisions when no one responds.



3. Scenario-Specific Workflow: Motor Vehicle Accident Claim

The insurance claim from lodging to closure. It covers policy verification, the parallel police check, damage inspection and repair quotes, fraud screening, approval by

amount, the repair or write-off decision, final inspection, payment and closing the claim.



Task 4 – Design Patterns

GoF Participants & Justifications

1.Strategy

Design problem: Different organisations assign and route work differently: by role, by workload, in turns, or by a rule calculated from data. If the choice were an if/else inside the engine, every new organisation would mean editing the engine.

Participants:

- Strategy: `AssignmentStrategy` (`selectParticipant`, `clone`, `name`)
- ConcreteStrategies: `RoleBasedStrategy` (the default), `ShortestQueueStrategy`, `RoundRobinStrategy`, `DataRuleStrategy`
- Context: `WorkAssigner`

Collaboration:

- When the coordinator releases a work item, it asks the `WorkAssigner` for a participant.
- The assigner reads the assignment rule named for that item in the definition (set with `buildAssignmentRule`). If none is named it uses the default role-based strategy.
- The strategy first keeps only the participants who hold the item's role, then applies its own rule: first match, lowest workload, next in turn, or a role calculated from instance data.
- The item is assigned to the chosen participant and an ASSIGNED event is published.

What varies: how a participant is chosen. The industry names the rule, and a new rule is a new class.

Why it fits: Assignment is business policy, so it must live in the definition and in small replaceable classes, not in the engine. In the insurance claim 'Verify police report' uses shortest-queue. In the bursary application 'Reference check' uses round-robin. `RoundRobinStrategy` remembers who was last, which is why strategies are objects with state and are cloned for each instance.

2.Mediator

Design problem: The parts of a running workflow have to coordinate, and without a mediator they would all depend on each other directly. Finishing one work item triggers several things: checking whether the next items' dependencies are met, evaluating their guards, routing approvals, assigning people and notifying them. In the insurance

scenario the police check, damage assessment and repair estimate run in parallel and the review must wait for all of them.

Participants:

- Mediator: `WorkflowMediator`
- ConcreteMediator: `InstanceCoordinator` (one per workflow instance)
- Colleague: `WorkflowColleague`
- ConcreteColleagues: `WorkItem`, `WorkAssigner`, `ApprovalHandler`

Collaboration:

- A colleague changes state and calls `notify(sender, event)` on its coordinator. It never calls another colleague.
- The coordinator first publishes the event to the observers, so the history is written even if a later reaction fails.
- On COMPLETED it records the completion and releases every item whose prerequisites are done. For each such item it evaluates the guard (Interpreter), skips the item or makes it available, routes its approval (Chain of Responsibility) and assigns it (Strategy).
- On REJECTED or CANCELLED it closes the item and cascades the closure to every item that depended on it.
- On a deadline passing it escalates the item, or moves an approval request to the next level.

What varies: the rules for what happens next, and which kinds of colleague exist.

Why it fits: The coordinator holds the instance's own progress (what is complete, closed or skipped), so two instances never interfere. The dependency and ordering rules live in one class, so a rule can change without editing the work-item classes.

3.Builder

Design problem: A workflow definition is a large structure of stages, work items, dependencies, guards, approval levels and external-system bindings that must be assembled in a valid order and checked before anything runs. Different sources need different amounts of checking, and the definition class should not be rewritten for each.

Participants:

- Builder: `WorkflowBuilder`
- ConcreteBuilder 1: `ValidatingWorkflowBuilder`

- ConcreteBuilder 2: `DraftWorkflowBuilder`
- Director: `WorkflowDirector`, with `MotorClaimWorkflowDirector` and `EducationBursaryWorkflowDirector`
- Product: `WorkflowDefinition`
- Client: `Scenario::install()`

Collaboration:

- The scenario creates a `ValidatingWorkflowBuilder` and passes it to its Director's `construct()`.
- The Director calls the build steps in a valid order: `setIdentity`, `buildStage`, `buildSubProcess`, `buildWorkItem`, `buildDependency`, `buildRule`, and the per-item steps `buildGuard`, `buildApproval`, `buildDecorations`, `buildService`, `buildAssignmentRule` and `buildEscalation`.
- The per-item steps record their settings in an `ItemConfig` for that item. All of it is plain text, so no engine class is involved.
- `getResult()` runs the final checks (empty or duplicate names, unknown items in a dependency, circular dependencies, items without roles) and returns the validated definition, which is stored in the `DefinitionRepository`.
- A definition from the `DraftWorkflowBuilder` is never marked validated, so the Facade refuses to start it.

What varies: how a definition is assembled, how strictly it is checked, and where its description comes from.

Why it fits: Construction logic is separated from the definition. Both industries use the same builders. Only the two Directors differ.

4.Prototype

Design problem: Many instances must start from one definition, and each needs its own progress, data and history. Changing one instance must never change the definition or another instance.

Participants:

- Prototype: `WorkflowComponent` (`clone()`)
- ConcretePrototypes: `WorkItem`, `Stage`, `SubProcess`
- Client: `WorkflowDefinition::createInstance()`
- Also cloneable: `Expression` and `AssignmentStrategy`

Collaboration:

- `createInstance(data)` calls `clone()` on the root of the template tree.
- A composite clones its children recursively, so the instance receives a fully separate copy. Every `WorkItem` copy starts in the Created state with fresh progress.
- The per-item settings (`ItemConfig`) are copied into the instance, together with fresh copies of the guard expressions and assignment strategies.
- The copy is wrapped in a new `WorkflowInstance` with its own id, data, coordinator, command processor and observers.

What varies: the kinds of work node and rule objects. A new type only needs its own `clone()`.

Why it fits: Instances come from copying a template, not rebuilding it, and the deep copy is what guarantees independence. Two claims started from 'motor-claim' progress separately.

5.Facade

Design problem: Starting and querying a workflow involves several subsystems (repository, definition, instance, engine). A user interface or an external trigger should not need to know the order or the details.

Participants:

- Facade: `WorkflowFacade`
- Subsystem classes: `DefinitionRepository`, `WorkflowDefinition`, `WorkflowInstance`, `WorkflowEngine`
- Clients: the text interface and `Workspace` (one facade per industry)

Collaboration:

- The client calls `startWorkflow(definitionId, data)`.
- The facade finds the definition (an error if it is unknown) and refuses it if it is not validated.
- It asks the definition to create an instance, registers the instance with the engine, starts it, and returns the instance id.
- The same facade offers `registerDefinition`, `addParticipant` and `getInstanceStatus`.

What varies: the internals of storage and the engine.

Why it fits: It gives TIVIDY one stable entry point, so the internals can be reorganised without breaking the interface or the tests.

6.Adapter

Design problem: TIVIDY must exchange information with outside systems whose interfaces differ and cannot be changed, for example `submit()` and `reverse()` on a bank, or a registry lookup. If the engine called each one directly, every workflow would contain vendor-specific code and each system would report failure differently.

Participants:

- Target: `ExternalConnector`
- Adapters (generic, reused by both industries): `PaymentAdapter`, `MessagingAdapter`, `RecordLookupAdapter`, `SchedulingAdapter`, `DocumentStorageAdapter`
- Adaptees: `PaymentAPI`, `MessagingAPI`, `RecordLookupAPI`, `SchedulingAPI`, `DocumentStoreAPI`
- Client: `WorkflowEngine`, through `InvokeServiceCommand` and `ConnectorRegistry`
- Supporting classes: `ConnectorRegistry`, `ConnectorRequest`, `ConnectorResult`

Collaboration:

- A work item is bound to a system in its definition with `buildService(item, connector, operation, params)`, for example the payment connector, the 'pay' operation and the parameters `account=$bankAccount;amount=$claimAmount*100`. A `serviceUndo` names how to reverse it.
- The engine runs an `InvokeServiceCommand`, which looks the connector up by name in the registry and sends a `ConnectorRequest`.
- The adapter checks the system is available, validates the request, calls the adaptee's own methods and converts the reply into a `ConnectorResult` (success, retryable, message, data).
- Success lets the workflow continue. A temporary failure is retried, and when retries run out an incident is raised. Undoing the command asks the connector to reverse (for example a refund).

What varies: which external systems exist, their interfaces and their operations.

Why it fits: Insurance registers policy-admin, police-system, payment and notify. Education registers student-records, income-verification, payment and notify. The payment and messaging adapters are shared as they are. The policy, police, student and income systems are all instances of the one generic `RecordLookupAdapter` over different data. This replaces the earlier insurance-specific adapter classes, so adding a system means registering a connector under a name, with no engine change.

7.Memento

Design problem: A running instance must be able to save its state at a checkpoint and return to it, without outside code reaching into its private details. The audit history must stay separate from the volatile runtime state.

Participants:

- Originator: `WorkflowInstance` (`createMemento()`, `restoreFromMemento()`)
- Memento: `InstanceMemento`, with the helper structs `WorkItemMemento` and `StageMemento`
- Caretaker: `ProcessDataStore` (one per instance, kept by `Session`)

Collaboration:

- Before each action in the interface, a checkpoint is saved: the caretaker appends the result of `instance->createMemento()`, a deep copy of the instance data, the state of every work item, the stage progress, the version and a timestamp.
- Only the instance can read or write the memento's contents. The caretaker never looks inside.
- On 'undo last step' the caretaker takes the latest memento and the instance calls `restoreFromMemento()`. Items and stages are reset and the coordinator rebuilds its progress records to match.
- The audit trail keeps every event, including the rollback itself.

What varies: the state recorded varies with the definition, but the classes are the same for every industry.

Why it fits: The earlier design used `InsuranceClaimInstance` and `ClaimInstanceMemento`, which tied the pattern to one industry. The generic classes serve both. A memento restores the engine's own records only. It cannot un-send a payment the bank already made. Reversing external effects is the job of Command compensation, so the two patterns work together.

8.Visistor

Design problem: Hard-coding reports and monitoring into workflow tasks bloats the code and forces edits to task classes for every new report.

Participants:

- Visitor: `WorkflowVisitor` (`visitInstance`, `visitStage`, `visitWorkItem`, `visitProcessDataStore`, `getReport`)
- ConcreteVisitors: `AuditReportVisitor`, `BottleneckVisitor`
- Element: `WorkflowElement`
- ConcreteElements: `WorkflowInstance`, `Stage`, `WorkItem`, `ProcessDataStore`
- Client: the text interface (progress reports menu)

Collaboration:

- The client creates a visitor and calls `instance->accept(visitor)`.
- The instance calls `visitor.visitInstance(*this)` and passes the visitor to the root of the Composite tree.
- Each `Stage` calls `visitStage(*this)` and passes the visitor to its children, so the visit follows the tree. Each `WorkItem` calls `visitWorkItem(*this)`. A decorator forwards the visitor to the item it wraps.
- The visitor builds its report as it goes. The audit report lists every stage and item with its state. The bottleneck report flags stages and items that exceeded their time limit.
- The client reads `getReport()`.

What varies: the reports and metrics.

Why it fits: A new report such as a cost or SLA report is one new visitor class and no change to the execution classes. This keeps work items and stages focused on routing.

9.Observer

Design problem: When something happens to a work item (it completes, fails, is escalated, skipped or refused), several independent parts of TIVIDY must react: the audit trail, the progress tracker, the notification service and the incident monitor. If the `WorkflowInstance` called each directly, it would be tightly coupled to all of them, and every new reaction would mean editing the instance.

Participants:

- Subject: `WorkflowSubject` (abstract). Keeps the observers and declares `attach()`, `detach()` and `notifyObservers(event)`.
- ConcreteSubject: `WorkflowInstance`. Its `raiseEvent(type, workItemId, cause, details)` builds a `WorkflowEvent` and notifies the observers.
- Observer: `WorkflowObserver` (abstract). Declares `update(event)` and the event types it subscribes to.
- ConcreteObservers:
    - `AuditLogger`: records every event with a timestamp as the audit trail.
    - `DependencyTracker`: keeps the lists of completed and closed items.
    - `NotificationService`: tells the participant who must act next, through a messaging connector.
    - `IncidentMonitor`: on FAILED asks the instance's `CommandProcessor` to retry the failed command. When retries run out it raises an incident.
- Event data: `WorkflowEvent` and `EventType` (CREATED, ASSIGNED, STARTED, COMPLETED, FAILED, REJECTED, CANCELLED, ESCALATED, COMPENSATED, TIMED_OUT, SERVICE_CALLED, SKIPPED, ROUTED, REFUSED). An event carries its type, instance, work item, time, details and the command that caused it.

Collaboration:

- Each observer is attached to the instance and subscribes to the event types it cares about.
- When something changes, the instance raises an event. `notifyObservers()` calls `update(event)` on each subscribed observer, passing the event as an argument (the push model).
- Each observer reacts independently. The audit logger writes a line, the notification service sends a message, the incident monitor asks for a retry.
- The coordinator publishes each event before it reacts to it, so the history is complete even if a later reaction fails.

What varies: the set of observers and the events they react to. A deadline monitor or a progress dashboard is a new subclass that is attached, with no change to the instance.

Why it fits: This is a one-to-many dependency in which the object that changes must not know who depends on it. When the assessor finishes the damage assessment, one event lets the audit logger record it, the tracker update progress and the notification service alert the next participant. Calling each directly would tie the instance to logging, messaging and failure handling.

10.Command

Design problem: Actions on a workflow, such as starting, assigning or completing a work item, or calling an external service like a payment gateway, must be recorded for the audit trail, retried if they fail temporarily, and undone in reverse order if a later step fails. A payout must be reversed when fraud is confirmed. If the engine performed these actions directly it would have no uniform way to log, retry or reverse them.

Participants:

- Command: `WorkflowCommand` (abstract). Declares `execute()` and `undo()`, and stores who issued it, when, its status (DONE, FAILED, UNDONE) and its attempt count.
- ConcreteCommands:
  - `StartWorkItemCommand` and `CompleteWorkItemCommand`: `undo()` of a completion reopens the item.
  - `AssignWorkItemCommand`: `undo()` restores the previous assignee.
  - `RejectWorkItemCommand` and `EscalateWorkItemCommand`.
  - `InvokeServiceCommand`: calls an external connector and keeps its reference. `undo()` asks the connector to reverse, for example a refund.
- Receivers: `WorkItem`, `Participant` and `ExternalConnector`.
- Invoker: `CommandProcessor` (one per instance). Runs and retries commands, keeps the history, and compensates.
- Client: `WorkflowEngine`. Creates the right command and hands it to the processor.

Collaboration:

- A person acts on an item in the interface. `ItemActions` maps the chosen action to an engine call, and the engine creates the matching command.
- The processor's `run()` calls `execute()` through the abstract interface. A completion goes through the outermost decorator, so security, validation and audit run first (see Decorator).
- The processor records each command with its status and attempts. A temporary failure is retried up to the retry limit.
- `compensate()` undoes every command that is DONE, newest first, and publishes a COMPENSATED event for each. Cancelling an instance after payment uses this to reverse the payment.
- Results are reported to the instance as events, which notify the observers.

What varies: the set of workflow actions and how each one is undone. A new action is a new command class and no change to the processor.

Why it fits: Command turns each request into an object, which is exactly what is needed to log it, retry it and reverse it. It supports the research finding that engines should retry temporary failures and use compensation for recovery. Each instance owns its own processor because the commands belong to one run.

11.Composite

Design problem: A workflow is not a flat list of tasks. It has single work items, stages that group them, and sub-processes that group stages. The engine must start, cancel, clone, visit and check progress on any of these without asking 'is this a task or a stage?'. Without that we would need exactly the type-checking and if/else chains the project rules ban.

Participants:

- Component: `WorkflowComponent` (abstract). The shared interface: `start()`, `complete()`, `cancel()`, `isComplete()`, `clone()`, `accept()`.
- Leaf: `WorkItem`. One unit of work done by a participant.
- Composites: `Stage` (holds children and applies operations to them according to its execution mode, sequential or parallel) and `SubProcess` (a nested group that sits inside another).
- Client: `WorkflowInstance`. Holds the root and drives it without knowing its shape.

Collaboration:

- The instance starts its root. A `Stage` passes operations to its children, which may be items or further stages.
- A parallel stage is finished when every child is finished, which is the join. A sequential stage moves on as each child ends.
- Which item becomes available when is decided by the dependencies held in the instance's coordinator. The stages group items and give them structure.
- An item whose branch is not taken is Skipped, and a skipped item counts as finished, so a stage or the whole instance can still complete.
- The prototype copy and the visitors traverse the same tree.

What varies: how big and how deeply nested a workflow is, and whether a group runs in sequence or in parallel. A new kind of group, such as a loop, would be a new composite and not an engine change.

Why it fits: The engine does the same job whether the target is a task or a whole process. In the insurance claim the police check, damage assessment, repair estimate and fraud screen run in parallel inside one stage, and 'Repair' is a nested sub-process, without the engine knowing.

12.State

Design problem: A work item moves through several states and what you may do depends on where it is. You cannot complete something that was never started or assign something cancelled. Checking this with if/else in every method would mean editing the same code each time a state or operation is added.

Participants:

- Context: `WorkItem`. Holds its current state and hands lifecycle calls to it.
- State: `WorkItemState` (abstract). Declares `makeAvailable()`, `assign()`, `start()`, `complete()`, `reject()`, `cancel()`, `escalate()`, `suspend()`, `resume()` and `skip()`, and refuses them all by default.
- ConcreteStates: `CreatedState`, `AvailableState`, `AssignedState`, `InProgressState`, `SuspendedState`, `EscalatedState`. Each allows only the operations valid at that point.
- Final ConcreteStates: `CompletedState`, `RejectedState`, `CancelledState` and the new `SkippedState`.

Collaboration:

- Something calls `start()` on a work item. The item checks nothing itself and calls `state->start(item)`.
- The current state either changes the item to the next state or throws an invalid-transition error.
- The engine turns that error into a REFUSED event, so the person sees, for example, 'Cannot start work item in state InProgress', and the audit trail records the refusal.
- Each state class has a shared `instance()`, so many items share a handful of state objects.
- When an item reaches a final state it notifies its coordinator, which releases or closes the items that depend on it.

What varies: which states exist, which transitions are legal, and what happens on entering or leaving a state.

Why it fits: The lifecycle is how TIVIDY knows what is done, what is active and what can happen next. Adding `SkippedState` for branches that are not taken needed one new class and a few changed transitions, with no edits to existing logic. Only `WorkItem` is stateful. A stage works out its status from its children.

13.Chain Of Responsibility

Design problem: An approval request may need to go through several levels. In the insurance scenario a claims handler can approve up to R20 000, a manager up to R100 000 and a senior manager anything above. In the bursary scenario the levels are Faculty Officer up to R10 000, Bursary Committee up to R50 000, Dean up to R150 000 and Finance Director with no limit. If the engine decided who approves with if/else on the amount, every new level or change of limit would mean editing the engine. If an approver does not respond in time the request must also move to the next level.

Participants:

- Handler: `ApprovalHandler` (abstract). Declares `handleRequest(request)`, holds a limit and a reference to its successor.
- ConcreteHandler: `LevelApprover`. One named level. A single class replaces the three fixed approver classes of the earlier design.
- Request: `ApprovalRequest`. Carries the amount, the work item, whether the deadline has passed, and who approved.
- Client: `InstanceCoordinator`, through `ApprovalRouter`, which parses the levels from configuration text and links them head to tail.

Collaboration:

- The definition holds the levels as text, for example `Claims Handler=20000;Claims Manager=100000;Senior Manager=*`, where * means no limit.
- When an approval item is released, the coordinator sends an `ApprovalRequest` into the first level.
- A level handles the request if the amount is within its limit and its deadline has not passed. Otherwise it passes the request to its successor with a fresh deadline.
- At the end of the chain the request stays with the top level, and the history records that no higher level exists.
- The chosen level is published as a ROUTED event and the item is assigned to someone holding that role.

What varies: the number of levels, their limits and their order, which differ between organisations and workflows.

Why it fits: Approval and escalation naturally form a chain in which each level handles the request or passes it on. The sender does not need to know who will finally approve. Levels belong to the definition, so the chain is built from configuration without changing the engine. Escalation on timeout is simply the request moving to the successor.

14.Decorator

Design problem: Some work items need extra behaviour on top of their normal lifecycle, such as validation before completion, priority handling, auditing, notifications or security checks. Different items need different combinations. A subclass for every combination would cause a class explosion, and putting all of it in `WorkItem` would bloat it.

Participants:

- Component: `WorkflowComponent` (abstract).
- ConcreteComponent: `WorkItem`. The basic unit of work with its normal lifecycle.
- Decorator: `WorkItemDecorator` (abstract). Is a `WorkflowComponent` and holds the component it wraps.
- ConcreteDecorators: `ValidationDecorator` (required data must be present before completing), `SecurityDecorator` (the acting participant must hold the required role), `AuditDecorator` (records each call), `NotificationDecorator`, `PriorityDecorator` (marks urgent items and shortens deadlines) and `DeadlineDecorator`.
- Assembler: `DecoratorFactory`, which reads the decoration text from the definition and wraps the item.

Collaboration:

- The definition says what an item needs in plain text, for example `audit,security:Finance Clerk`, using `buildDecorations`.
- When an instance starts, `DecoratorFactory` wraps each item in that order. The first name is the innermost layer and the last is the outermost. The instance returns the outermost layer through `frontOf(itemId)`.
- The engine's commands call the outermost layer, so every decorator really runs. Each does its extra work and passes the call inward.
- The call finally reaches the `WorkItem`, which changes state through its State object.

- To the rest of the engine a decorated item is still just a `WorkflowComponent`.

What varies: which extra behaviours an item has and in what combination. A new behaviour, such as an SLA timer, is a new decorator.

Why it fits: In the insurance claim 'Release settlement payment' is wrapped with audit and security (only a Finance Clerk may act), and 'Collect claim documents' is wrapped with validation (the police case number must exist). `WorkItem` stays focused on its lifecycle and each extra behaviour is in its own small class.

15.Interpreter

Design problem: Routing decisions such as 'only investigate fraud if the claim looks suspicious' or 'merit bursaries take a different route to need bursaries' differ between industries and between processes. They must be configurable without editing the engine, and written in the definition like the rest of the process.

Participants:

- AbstractExpression: `Expression` (`evaluate(context)`, `clone()`, `toString()`)
- TerminalExpression: `ComparisonExpression`, one comparison such as a data value compared with a number, or the outcome of an earlier item compared with a word
- NonterminalExpressions: `AndExpression`, `OrExpression`, `NotExpression`
- Context: `EvaluationContext`, which looks up instance data and the outcomes of earlier items
- Client: `ExpressionParser` (turns text into an expression tree) and `InstanceCoordinator` (evaluates it)

Collaboration:

- A definition attaches a guard to an item with `buildGuard`. For example 'Authorise repair' carries the guard `outcome('Review claim and decide') == REPAIR`.
- The text is parsed once into a tree of `Expression` objects, and each instance receives its own clone.
- When the item's prerequisites are done, the coordinator evaluates the tree with the instance data as context.
- True: the item is routed on and a ROUTED event records the branch taken. False: the item is Skipped and a SKIPPED event records which rule was not met.

What varies: the rules themselves, per industry and per process.

Why it fits: Insurance guards investigate fraud only if the screening outcome is SUSPICIOUS or the fraud score is high, and choose the repair path or the write-off path from the review decision. Education guards choose the merit or need route and waitlist an applicant when the fund is too small. None of this is an if/else in engine code.

16.Template Method

Design problem: Installing an industry always follows the same order: build and validate the workflow definition, create the participants, register the connectors to external systems. Each industry differs in the content of those steps but must never forget one or do them in the wrong order.

Participants:

- AbstractClass: `Scenario`, whose `install()` is the template method
- Primitive steps: `director()`, `addParticipants()` and `addConnectors()`
- ConcreteClasses: `InsuranceScenario` and `EducationScenario`

Collaboration:

- `install()` creates a `ValidatingWorkflowBuilder`, calls `director().construct(builder)` and stores the validated definition in the repository.
- It then calls `addParticipants()` to create the people and roles, and `addConnectors()` to register the stub external systems.
- The concrete scenario supplies only the content of each step.

What varies: the process, the people and the external systems of each industry.

Why it fits: Everything that makes an industry different lives in the `scenarios` folder. Each industry is installed the same way, so a missing connector is caught at set-up and not when a payment is first attempted.

17.Factory Method

Design problem: The interface must offer 'the available industries' and work with one of them without naming `InsuranceScenario` or `EducationScenario`. A new industry should appear in the menu without the interface changing.

Participants:

- Creator: `ScenarioCatalog` (`ids()` and `create(id)`)
- Product: `Scenario`
- ConcreteProducts: `InsuranceScenario`, `EducationScenario`
- Client: `Session`

Collaboration:

- `Session` asks the catalog for the list of scenario ids.
- For each id it calls `ScenarioCatalog::create(id)` and receives a `Scenario` that it knows only through the abstract interface.
- It builds one `Workspace` per scenario, and each scenario installs itself (Template Method).

What varies: which industries exist.

Why it fits: Adding a third industry means one new `Scenario` subclass and one line in the catalog. The engine, the Facade and the text interface do not change. Note that this is a static factory method and not the subclass-overriding form of the pattern, so describe it that way in the report.

Design Desions and revisions since practical 6

Revision 1: industry rules became configuration text.

- Problem: Approval limits, branches, decorations and system bindings were spread between code and demonstration scripts, so a new industry would have needed engine edits.
- Original design: Rules held in classes and in the demonstration code.
- Change: Each work item carries an `ItemConfig` written by the industry's Director through `buildGuard`, `buildApproval`, `buildDecorations`, `buildService`, `buildAssignmentRule` and `buildEscalation`. The engine only reads it.
- Why: One engine runs two industries, and the engine contains no industry words.
- Diagrams to update: UML class diagram.

Revision 2: approval levels come from text (Chain of Responsibility).

- Problem: Three fixed approver classes hard-coded the insurance levels.
- Change: One `LevelApprover` class and an `ApprovalRouter` that builds the chain from text such as `Name=limit;Name=limit;Name=*`.
- Why: The bursary has four levels with different limits, and a new level should not need a new class.

- Diagrams to update: UML class diagram, approval and escalation activity diagram.

Revision 3: decorators sit in the real execution path.

- Problem: Decorators existed but the engine acted on the inner work item directly.
- Change: `DecoratorFactory` wraps items from the definition, `frontOf()` returns the outermost layer, and commands call that layer.
- Why: Security, validation and audit must run on every action.
- Diagrams to update: UML class diagram, completion sequence diagram.

Revision 4: adapters are all generic (replaces the Practical 6 split).

- Problem: Insurance-specific adapter classes tied part of the engine's surroundings to one industry.
- Change: Policy, police, student and income systems are instances of one `RecordLookupAdapter` over different data. Payment and messaging adapters are shared by both industries.
- Why: A new system is a registered connector, not a new class.
- Diagrams to update: UML class diagram, adapter class diagram.

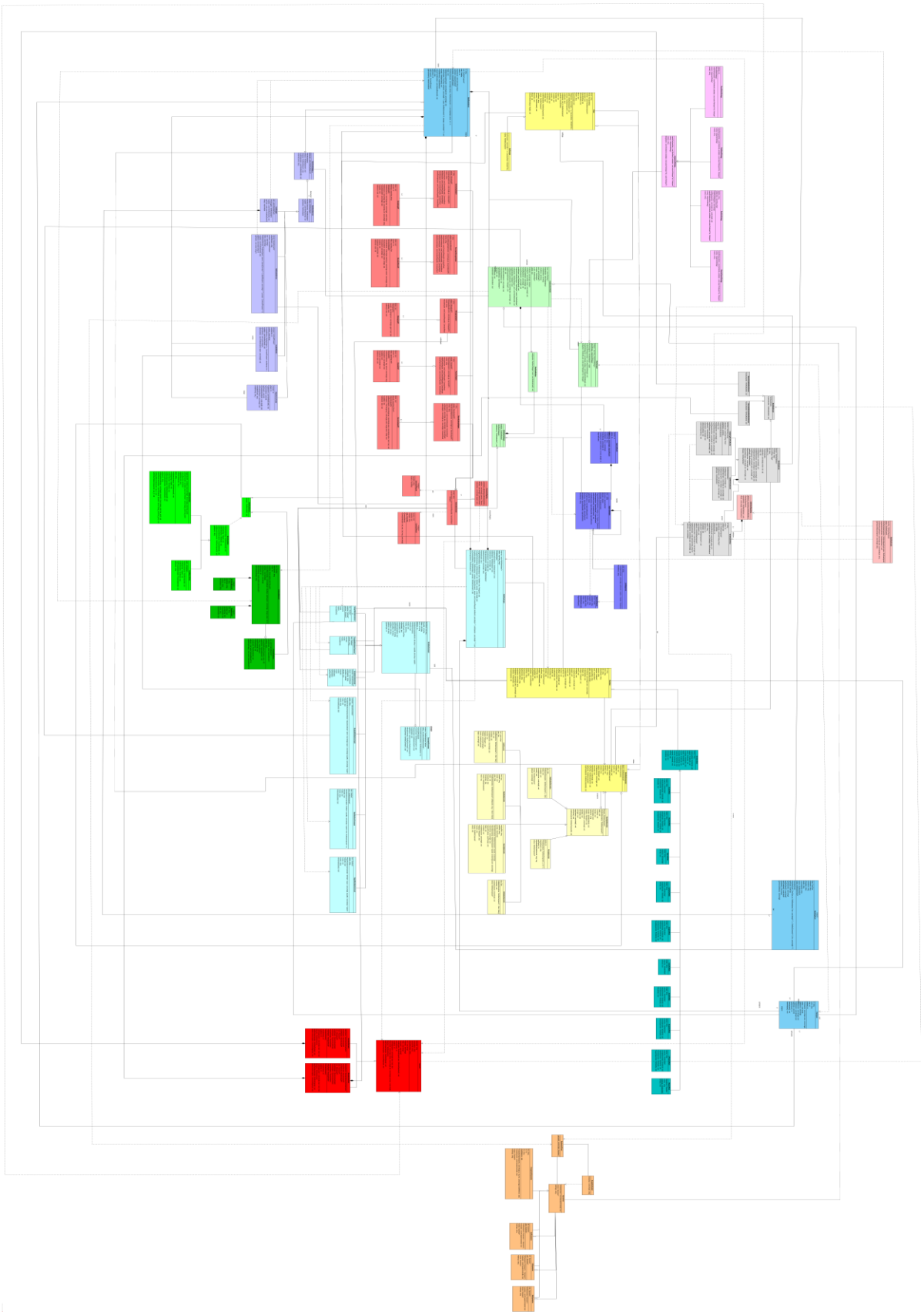
Revision 5: Memento and Visitor made generic.

- Problem: `InsuranceClaimInstance` and `ClaimInstanceMemento` tied Memento to one industry, and the Visitor listed only `WorkflowInstance` as an element.
- Change: `WorkflowInstance`, `InstanceMemento` and `ProcessDataStore` are used for every industry. The Visitor elements are `WorkflowInstance`, `Stage`, `WorkItem` and `ProcessDataStore`.
- Diagrams to update: UML class diagram.

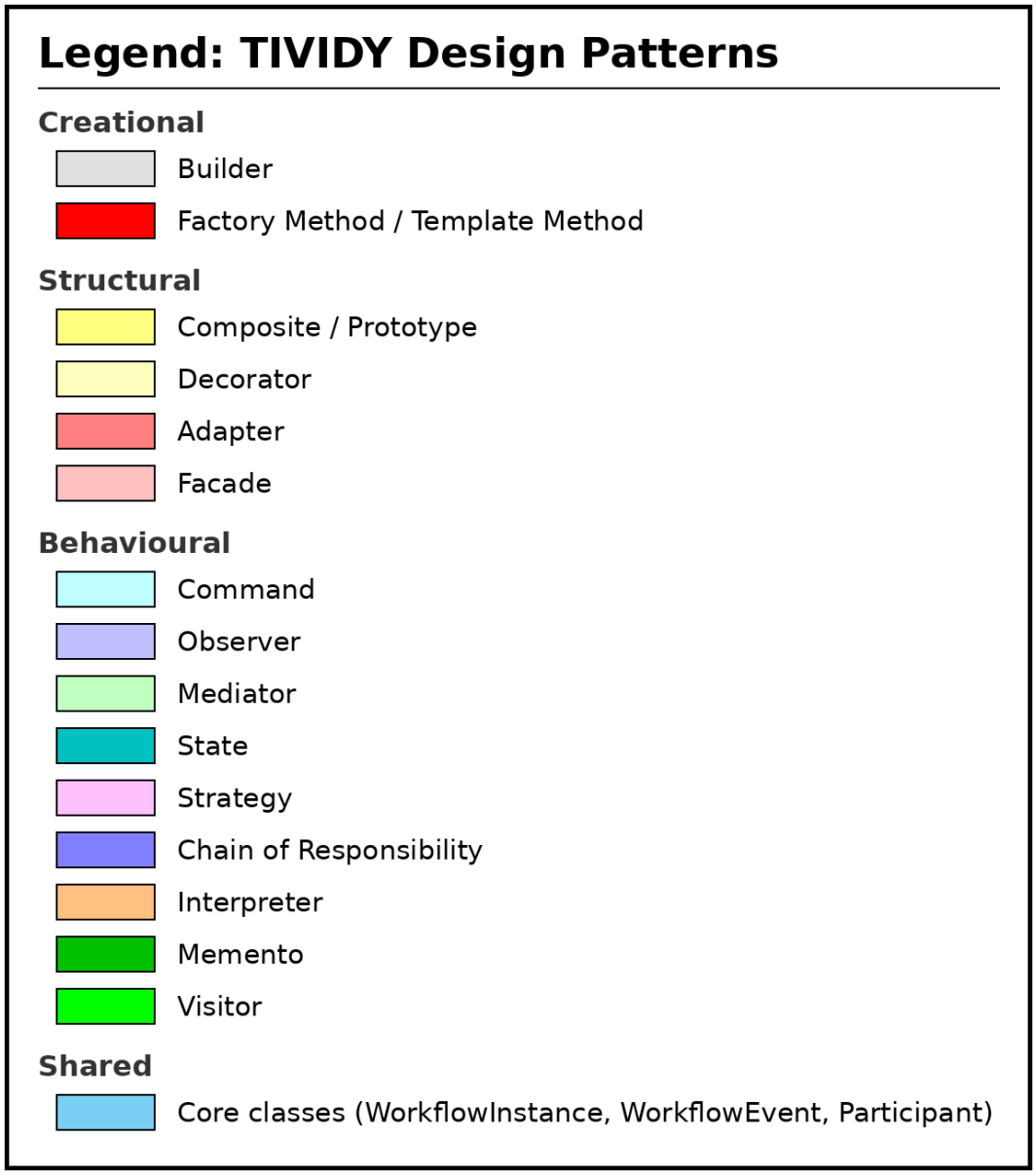
Revision 6: guards, skipping and scenario classes added.

- Problem: Branching (repair or write-off, merit or need) and industry set-up had no home in the design.
- Change: The Interpreter evaluates guards, `SkippedState` records a branch not taken, `Scenario::install()` fixes the set-up order and `ScenarioCatalog` creates scenarios.

TASK5:UML DIAGRAM



UML LEGEND



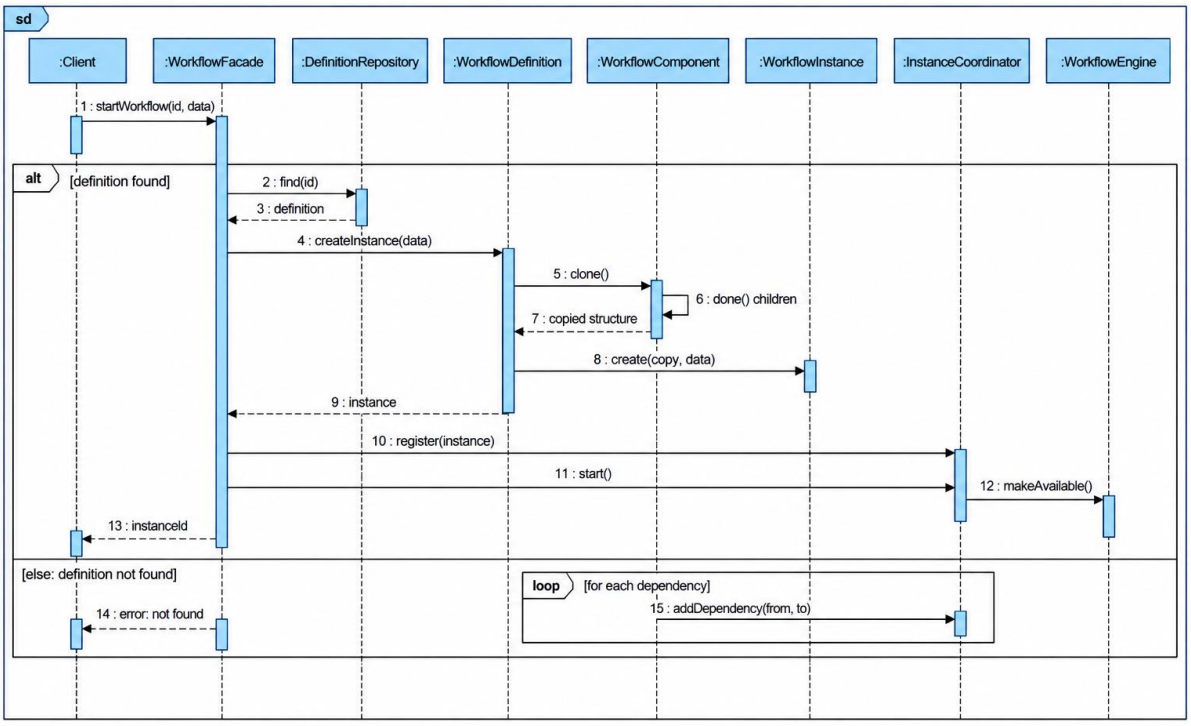
Click for a clearer version of this class diagram:

TIVIDY UML class diagram: high-resolution version

Task 6: Runtime behaviour diagrams

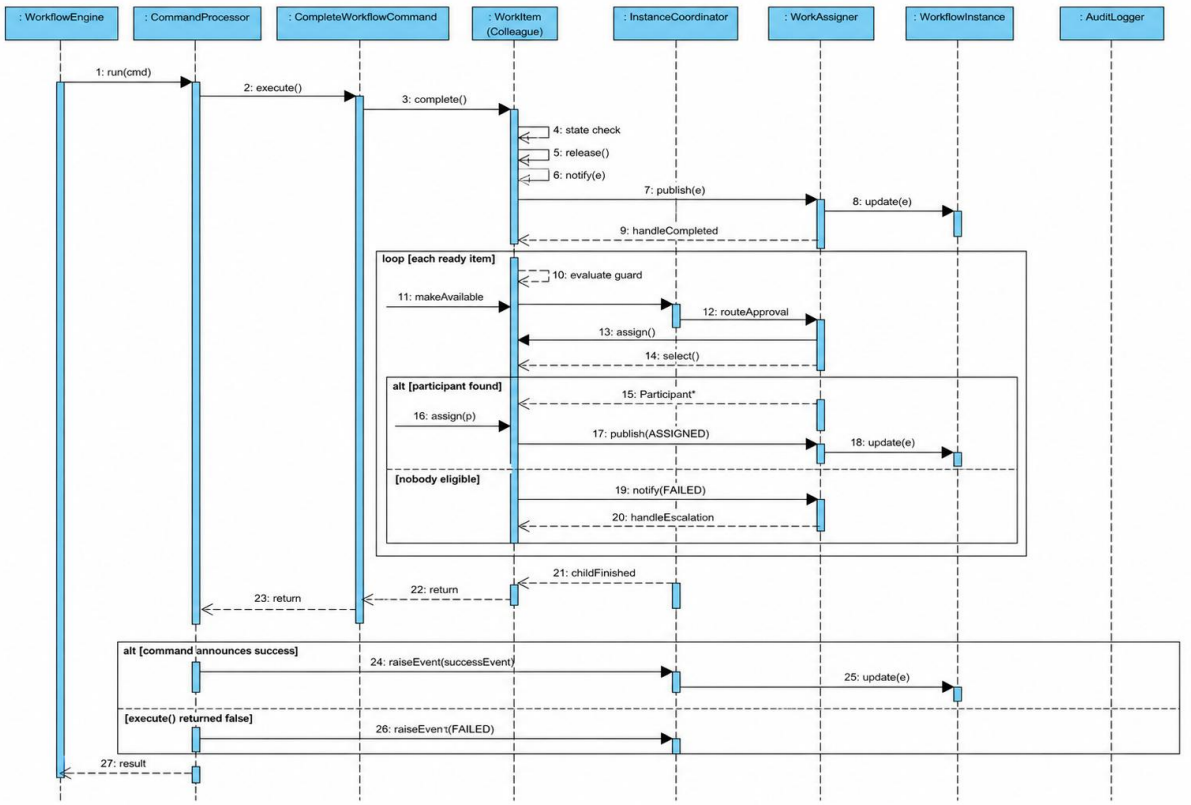
Sequence Diagram: Definition and Instance Creation

How a client starts a workflow through the WorkflowFacade. The definition is found in the repository, its structure is cloned into a new independent instance, and the instance is registered and started with the engine. If the definition is not found, an error is returned.



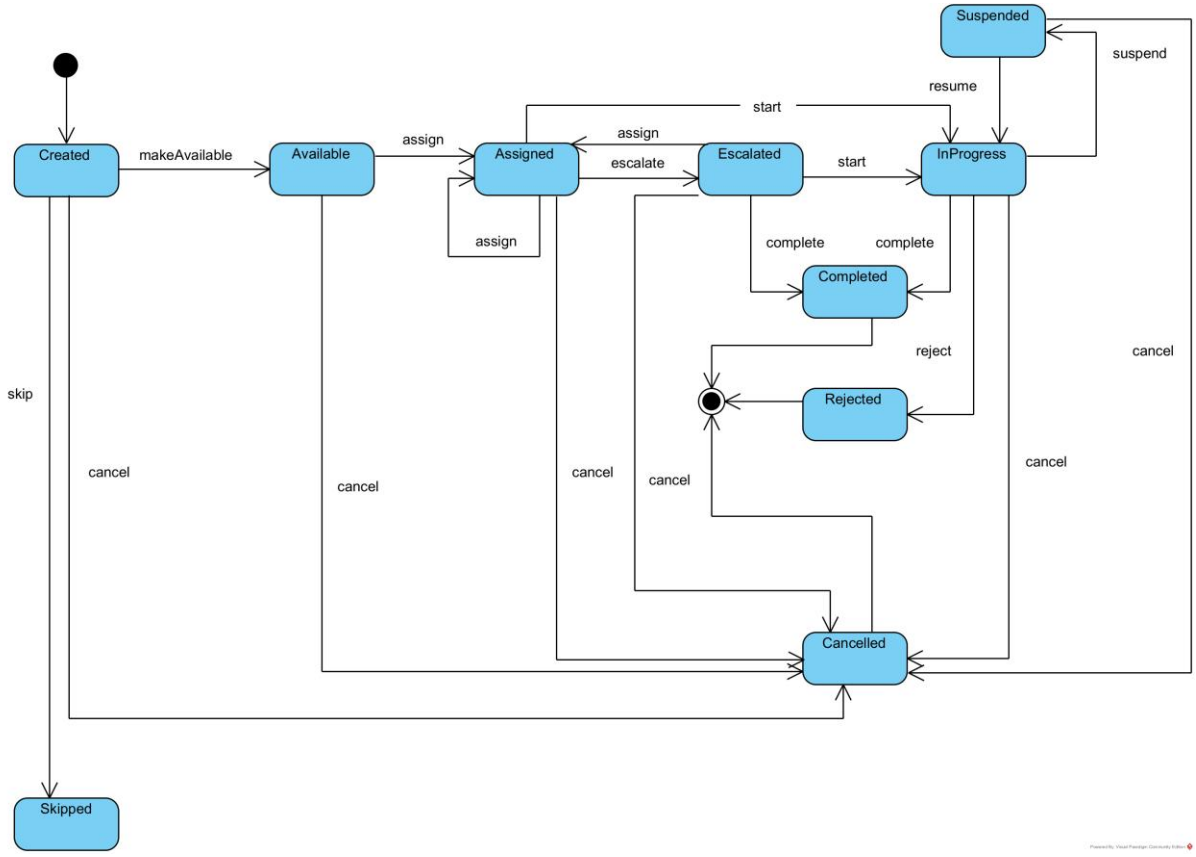
Sequence Diagram: Completing a Work Item

How the WorkflowEngine completes a work item through the CommandProcessor using a CompleteWorkItemCommand. The InstanceCoordinator is notified, makes the next work item available and assigns it through the WorkAssigner. The WorkflowInstance then raises a COMPLETED event, and the AuditLogger records it.



State Diagram: Work Item Lifecycle

The states a work item moves through: Created, Available, Assigned, Escalated, InProgress and Suspended, ending in Completed, Rejected, Skipped or Cancelled. Each arrow is an operation (makeAvailable, assign, start, skip, escalate, suspend, resume, complete, reject, cancel) that is only allowed from certain states.



TASK 7: Design Decisions and Revisions

Revision 1: Adapters split into generic and industry-specific (Adapter)

Problem: The final project requires one reusable engine running two industries. Adapters named after insurance systems would have tied part of the engine to insurance, and the same payment and messaging code would have been rewritten for the second industry.

Question being considered: Which external-system adapters can both industries reuse unchanged, and which belong to one industry?

Original design or assumption: One adapter per external system in the insurance scenario (policy administration, police verification, vehicle valuation, panel beater portal, payment gateway, SMS and email), each wrapping its own adaptee. All of them were treated as part of the engine.

Change made: The adapters were separated into two groups. The generic ones (PaymentAdapter, MessagingAdapter, SchedulingAdapter, DocumentStorageAdapter) wrap a technology and know nothing about any industry, so both industries reuse them without modification. The insurance-specific ones (PolicyAdminAdapter, PoliceVerificationAdapter, ValuationAdapter, PanelBeaterPortalAdapter) moved into a scenario folder outside the engine. ConnectorRegistry was added so the coordinator finds a connector by a configured name and never names a specific system. A generic ConnectorRequest (an operation name and a map of parameters) and ConnectorResult (success, retryable, message and data) are used by every adapter.

Diagrams to be updated: UML Class Diagram, Adapter pattern class diagram, Activity Diagram 1.

Revision 2: industry rules became configuration text.

- Problem: Approval limits, branches, decorations and system bindings were spread between code and demonstration scripts, so a new industry would have needed engine edits.
- Original design: Rules held in classes and in the demonstration code.
- Change: Each work item carries an `ItemConfig` written by the industry's Director through `buildGuard`, `buildApproval`, `buildDecorations`, `buildService`, `buildAssignmentRule` and `buildEscalation`. The engine only reads it.
- Why: One engine runs two industries, and the engine contains no industry words.

- Diagrams to update: UML class diagram.

Revision 3: approval levels come from text (Chain of Responsibility).

- Problem: Three fixed approver classes hard-coded the insurance levels.
- Change: One `LevelApprover` class and an `ApprovalRouter` that builds the chain from text such as `Name=limit;Name=limit;Name=*`.
- Why: The bursary has four levels with different limits, and a new level should not need a new class.
- Diagrams to update: UML class diagram, approval and escalation activity diagram.

Revision 4: decorators sit in the real execution path.

- Problem: Decorators existed but the engine acted on the inner work item directly.
- Change: `DecoratorFactory` wraps items from the definition, `frontOf()` returns the outermost layer, and commands call that layer.
- Why: Security, validation and audit must run on every action.
- Diagrams to update: UML class diagram, completion sequence diagram.

Revision 5: adapters are all generic (replaces the Practical 6 split).

- Problem: Insurance-specific adapter classes tied part of the engine's surroundings to one industry.
- Change: Policy, police, student and income systems are instances of one `RecordLookupAdapter` over different data. Payment and messaging adapters are shared by both industries.
- Why: A new system is a registered connector, not a new class.
- Diagrams to update: UML class diagram, adapter class diagram.

Revision 6: Memento and Visitor made generic.

- Problem: `InsuranceClaimInstance` and `ClaimInstanceMemento` tied Memento to one industry, and the Visitor listed only `WorkflowInstance` as an element.
- Change: `WorkflowInstance`, `InstanceMemento` and `ProcessDataStore` are used for every industry. The Visitor elements are `WorkflowInstance`, `Stage`, `WorkItem` and `ProcessDataStore`.
- Diagrams to update:Uml diagram

Revision 7: guards, skipping and scenario classes added.

- Problem: Branching (repair or write-off, merit or need) and industry set-up had no home in the design.
- Change: The Interpreter evaluates guards, `SkippedState` records a branch not taken, `Scenario::install()` fixes the set-up order and `ScenarioCatalog` creates scenarios.

The PDF should contain, in a sensible order, at least: • team details and GitHub repository link; • workflow-management research findings and references; • selected scenario description; • initial TIVIDY system scope and design rationale; • the three UML Activity Diagrams; • complete UML Class Diagram; • GoF participant mapping and pattern justifications; • two UML Sequence Diagrams; • one UML State Diagram; • important design decisions and revision history; and • ashort team contribution statement

TEAM CONTRIBUTION STATEMENT

Lizalise Mbonisweni

- *Practical 6:* Did theme 4 of the research, designed the Chain of Responsibility and Decorator patterns, drew her part of the UML class diagram and one of the activity diagrams.
- *Final project:* Implemented the approval chain and the workflow decorators, including the ApprovalDecorator that connects the decorator stack to the approval chain and a SecurityDecorator that allows system cancels. Wrote the Chain of Responsibility and Decorator tests and the pattern integration tests (Composite, Decorator, Chain and Visitor). Drew the bursary (education) scenario activity diagram.

Boikemelo Masoka

- *Practical 6:* Did theme 5 of the research, designed the Observer and Command patterns, did tasks 1 and 2 with Reneilwe, Dingalethu and Thembelisha, and drew her part of the UML class diagram.
- *Final project:* Implemented the Command pattern (WorkflowCommand, the concrete commands and the CommandProcessor) and the Observer pattern (WorkflowSubject, WorkflowObserver, AuditLogger, NotificationService and IncidentMonitor, using a push model). Implemented the WorkflowEngine, including retries, compensation on cancel and undo. Wrote the Command, Observer and WorkflowEngine tests and the final integration test. Combined the final project UML class diagram and organised the diagrams folder.

Thembelisha Skosana

- *Practical 6:* Did theme 7 of the research, designed the Memento and Visitor patterns, did tasks 1 and 2 with Reneilwe, Dingalethu and Boikemelo, drew her part of the UML class diagram, and combined the Practical 6 UML class diagram with Lethabo.
- *Final project:* Implemented the Memento pattern (InstanceMemento and the ProcessDataStore caretaker) and the Visitor pattern (audit report and bottleneck visitors), and connected the two. Set up the Doxygen API documentation and its main page, documented the State, Composite and Visitor interfaces, and wrote the README.
- Built the text interface (TUI) with Dingalethu and Lethabo.

Lethabo Molobi

- *Practical 6:* Did theme 6 of the research, designed the Adapter pattern, drew one of the three activity diagrams, drew her part of the UML class diagram, and helped Thembelisha combine the Practical 6 UML class diagram.
- *Final project:* Implemented the Adapter pattern: the ExternalConnector interface, the connector registry, and the payment, messaging, scheduling, document storage and record lookup adapters with their simulated external systems. Wrote the adapter unit tests. Fixed cancelling after payment (system-acting state during cancel) and added a regression test for it.
- Built the text interface (TUI) with Dingalethu and Thembelisha.

Boitumelo Monareng

- *Practical 6:* Did theme 3 of the research, designed the Mediator and Strategy patterns, drew one of the three activity diagrams and one of the two sequence diagrams, and drew her part of the UML class diagram.
- *Final project:* Implemented the Mediator pattern (WorkflowMediator and InstanceCoordinator) and the Strategy pattern (WorkAssigner and the role-based, shortest-queue, round-robin and data-rule strategies), plus the shared Participant class. Wrote the Mediator and Strategy tests. Set up the GitHub Actions CI workflow and the Makefile.

Reneilwe Molopyane

- *Practical 6:* Did theme 1 of the research, designed the Builder, Facade and Prototype patterns, did tasks 1 and 2 with Thembelisha, Dingalethu and Boikemelo, drew her part of the UML class diagram and one of the sequence diagrams.
- *Final project:* Implemented the Builder pattern (WorkflowBuilder, ValidatingWorkflowBuilder, DraftWorkflowBuilder, WorkflowDirector and WorkflowDefinition), the Facade (WorkflowFacade and DefinitionRepository) and Prototype instance creation. Wrote the builder, definition, director, facade and repository tests and the integration tests. Wrote the all-patterns demo program and its Makefile target.

Dingalethu Ngumbela

- *Practical 6:* Did theme 2 of the research, designed the Composite and State patterns, did tasks 1 and 2 with Thembelisha, Reneilwe and Boikemelo, and drew his part of the UML class diagram and the state diagram.
- *Final project:* Implemented the Composite pattern (WorkflowComponent, Stage, SubProcess and WorkItem) and the State pattern (WorkItemState, the concrete states and InvalidTransitionException). Integrated all the patterns into the final

engine. Added the insurance and education scenarios (Factory Method and Template Method), the Interpreter for routing rules, the text interface (TUI), and the web app with its API. Added the Dockerfile and scenario integration tests, and updated the state diagram.

- Built the text interface (TUI) with Thembelisha and Lethabo.